VISUAL SPEAKER NOTES • CONTENT-DRIVEN EDITION

Visual Speaker Notes — /retro in Practice

Matt Pocock's New Skill, Clearly Explained (with real sessions)

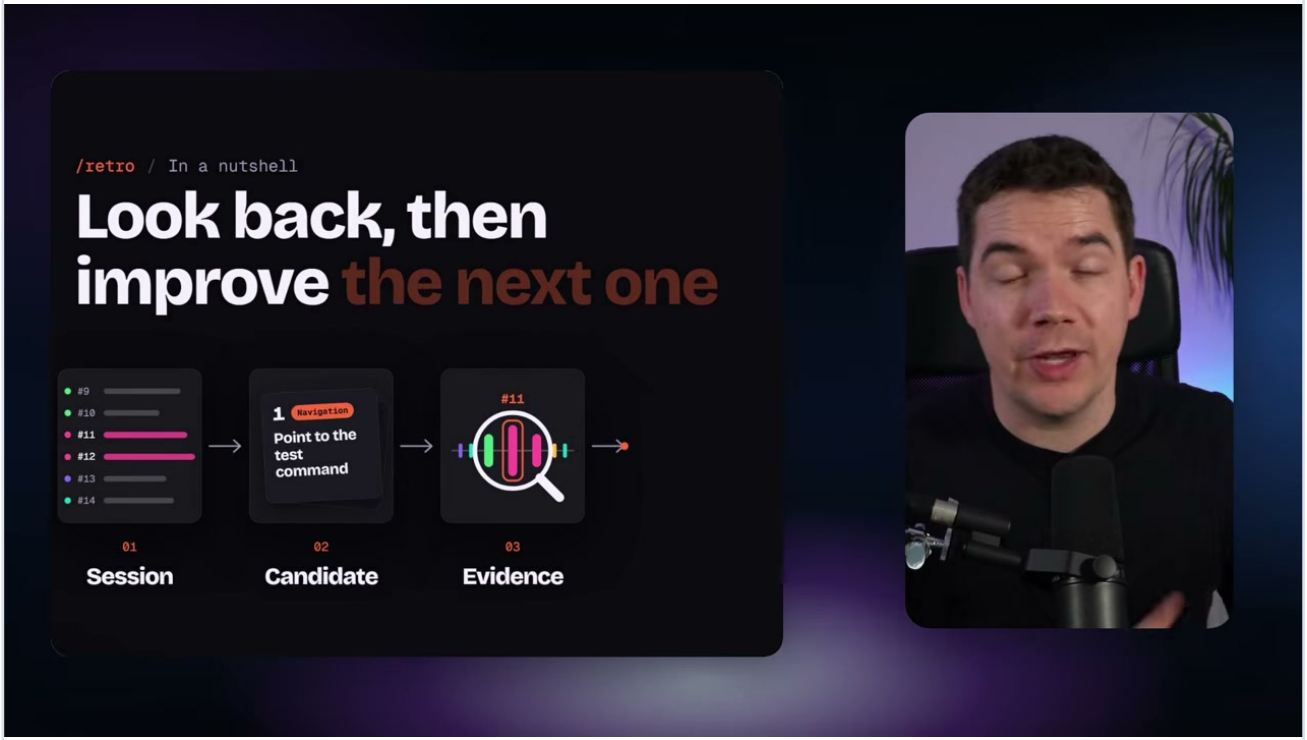
The AI Automators • 2026-10-06 • 08:38

- 22 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Speaker claims and caveats belong in the notes

https://www.youtube.com/watch?v=7nNJZmyBlm0

/retro in a Nutshell

Look back at a finished session to improve the next one



SPEAKER NOTES

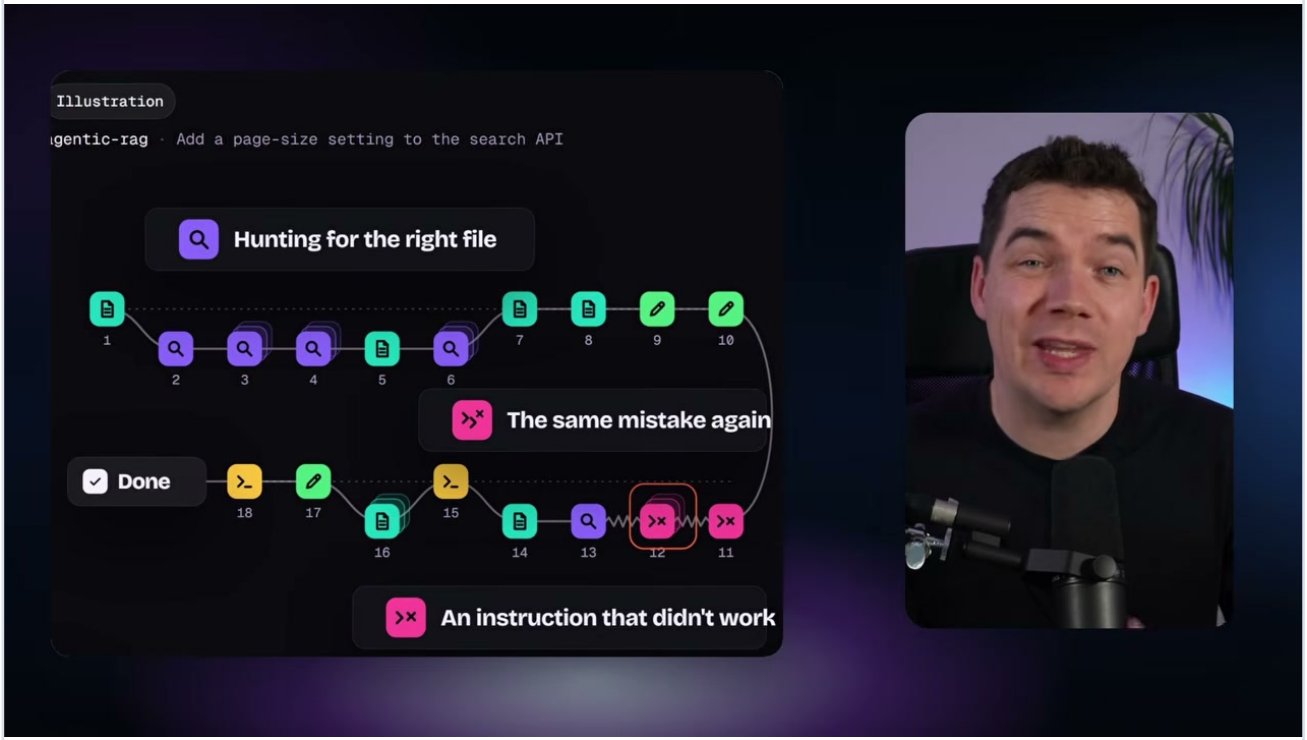
- The skill reviews a completed coding-agent session rather than judging only the final code.
- It turns session behavior into candidate improvements backed by evidence.
- The user decides which candidates are worth acting on.

WHY IT MATTERS

A retrospective converts hidden execution friction into concrete opportunities for the next run.

Read the Path, Not Just the Result

Session events reveal repeated mistakes and wasted motion



SPEAKER NOTES

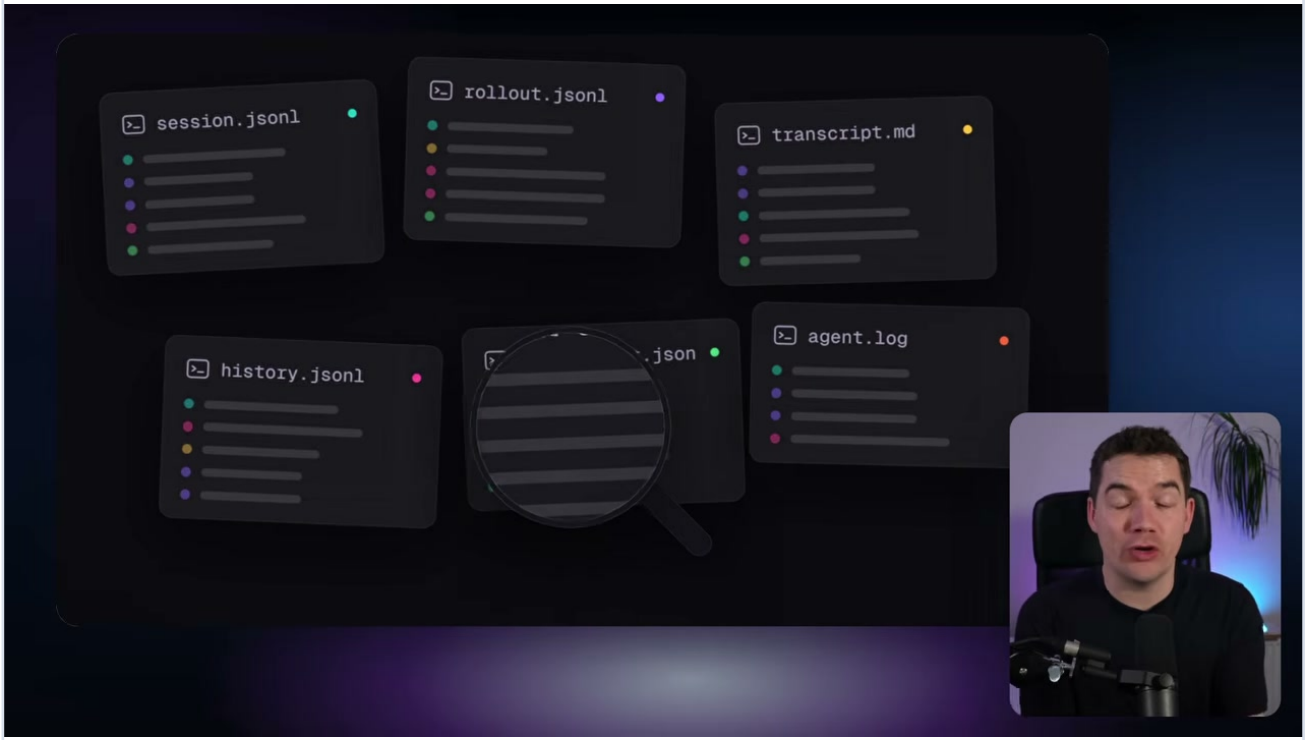
- The illustrated trace marks file hunting, a repeated mistake, an ineffective instruction, and eventual completion.
- These problems can disappear from view once the agent reports that the task is done.
- The trace supplies evidence for ranking possible improvements.

WHY IT MATTERS

The route to a result can expose costs and risks that the finished diff does not show.

What /retro Reads

Saved session artifacts become the evidence layer



SPEAKER NOTES

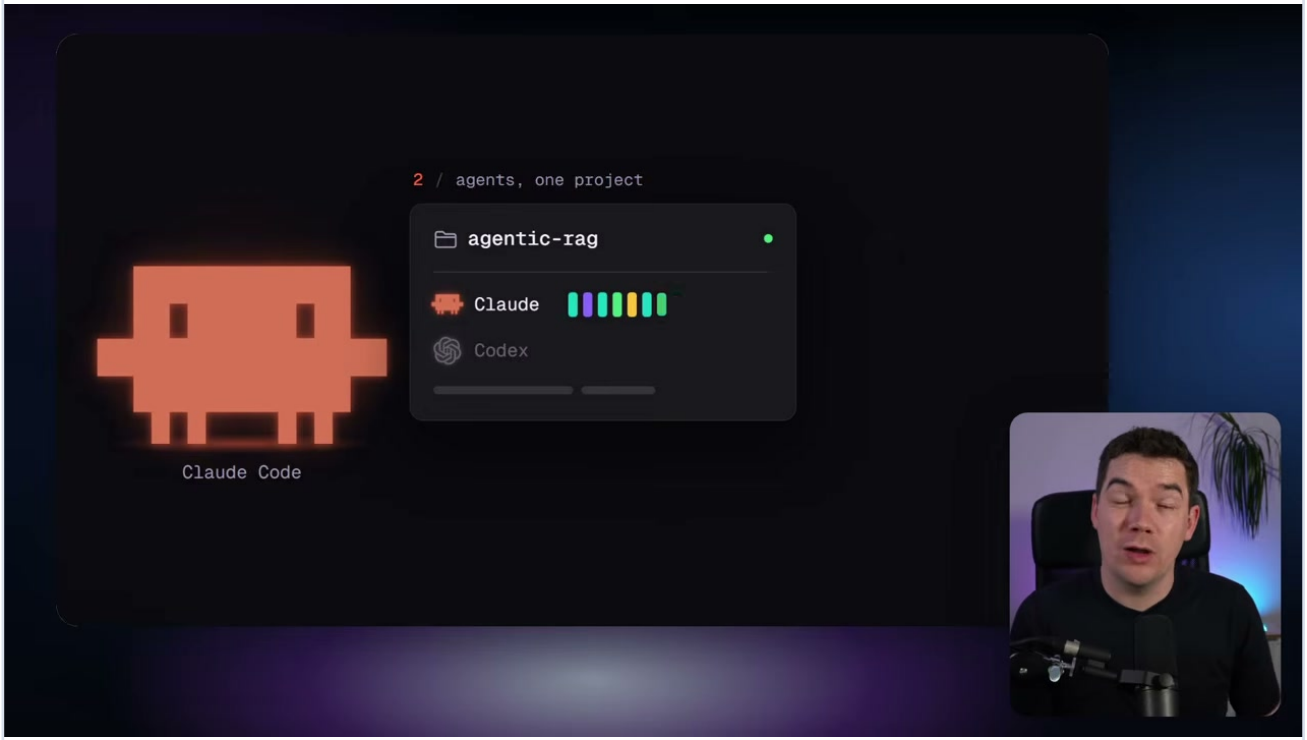
- The visual shows session, rollout, transcript, history, chat-export, and agent-log artifacts.
- The skill inspects these records for recurring errors, unnecessary work, and failed instructions.
- The presenter describes passing the relevant session material to an agent that follows the skill's review procedure.

WHY IT MATTERS

Structured session records make agent behavior inspectable after the coding task has ended.

Two Agents, One Project

Claude Code and Codex reviewed the same agentic-rag codebase



SPEAKER NOTES

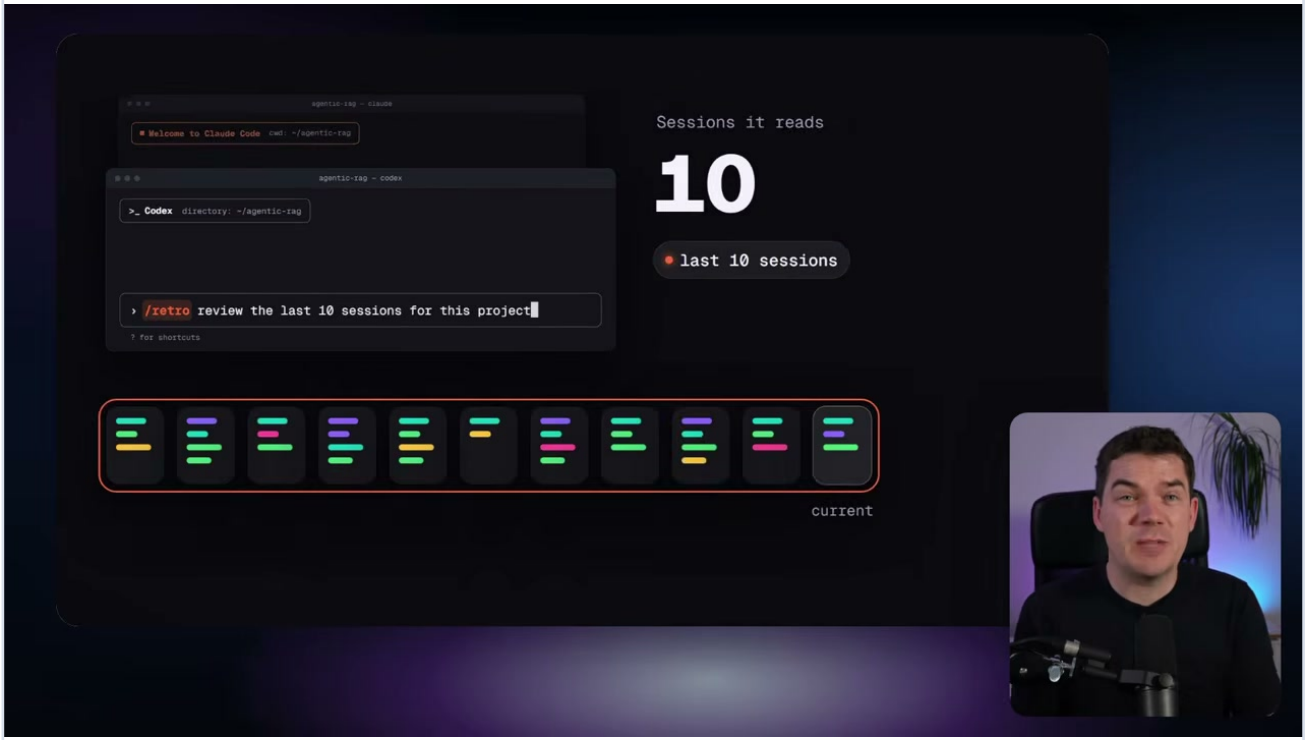
- The experiment used the full-stack agentic-rag project, which includes chat, citations, ingestion, skills, and an LLM wiki.
- Both Claude Code and Codex had recent work histories on that project.
- This is a project-specific field sample, not a controlled cross-model benchmark.

WHY IT MATTERS

Comparing independent session histories can reveal project friction that is not tied to one agent.

The 10 + 10 Session Experiment

Review the last ten sessions in each coding tool



SPEAKER NOTES

- The presenter asked /retro to review the last 10 project sessions in Claude Code and the last 10 in Codex.
- The session range had to be specified because the skill reviews the current session by default.
- The two reports were combined to look for recurring themes.

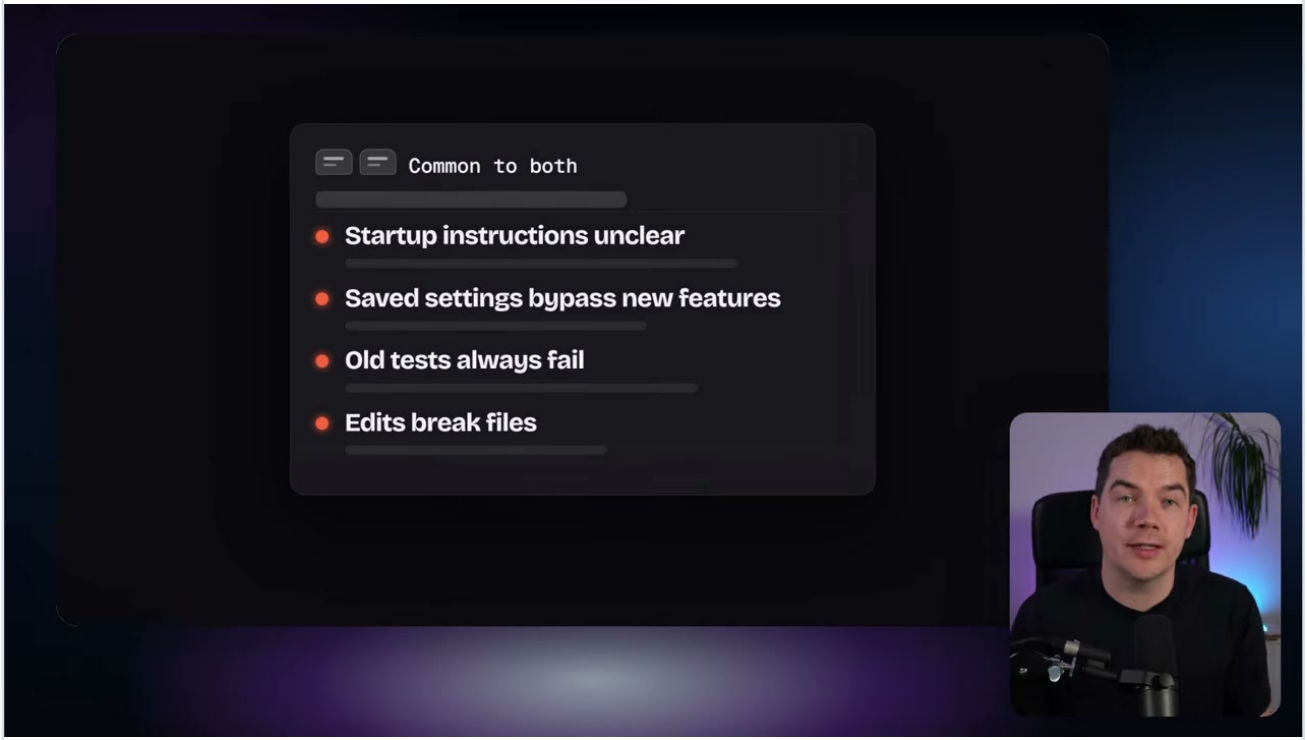
WHY IT MATTERS

A defined sample makes the review reproducible while keeping conclusions scoped to those sessions.

Source: https://www.youtube.com/watch?v=7nNJZmyBlm0

Four Findings Common to Both

Recurring friction surfaced across the sampled histories



SPEAKER NOTES

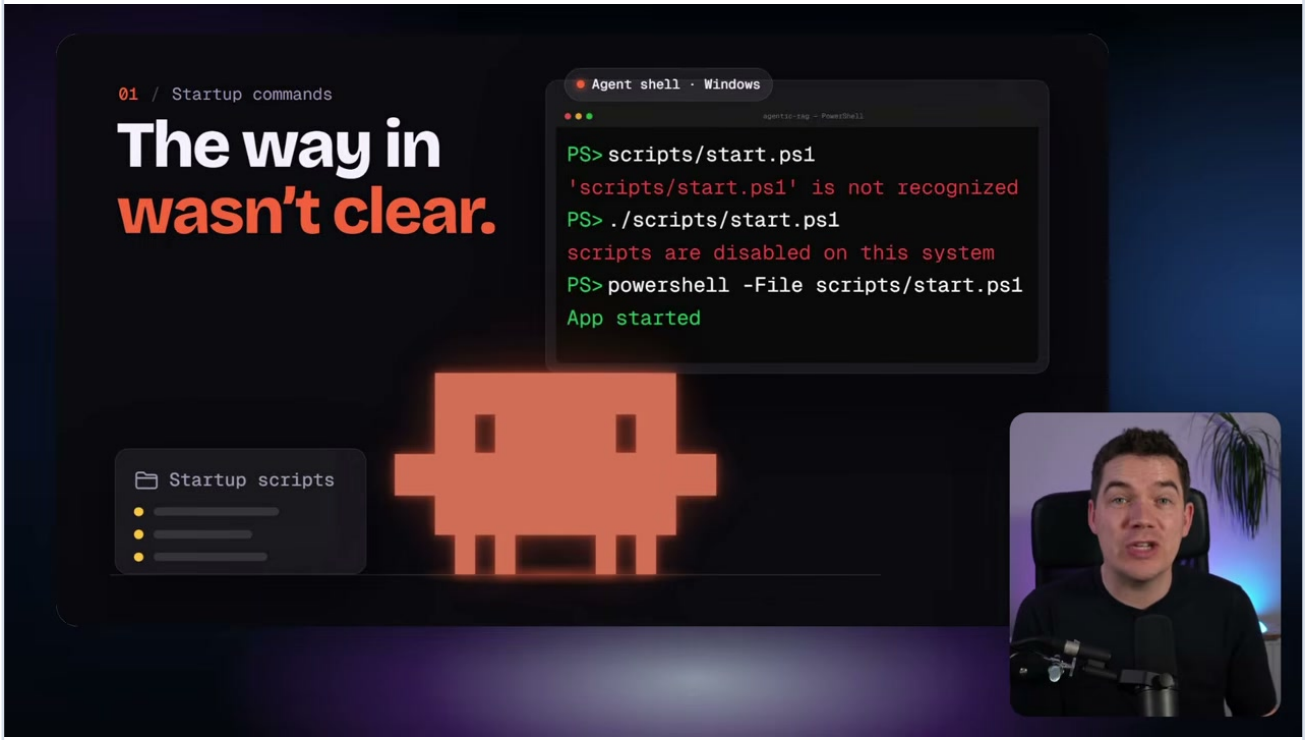
- The shared findings were unclear startup instructions, saved settings bypassing new features, old tests always failing, and edits breaking files.
- Agreement across both reports made these candidates worth investigating first.
- These are observed findings from this project and 20-session sample, not universal claims about Claude Code or Codex.

WHY IT MATTERS

Repeated, cross-agent friction points to environment and codebase changes with potentially broad payoff.

Finding 1 — The Startup Route Was Unclear

Windows agents repeatedly rediscovered the working command



SPEAKER NOTES

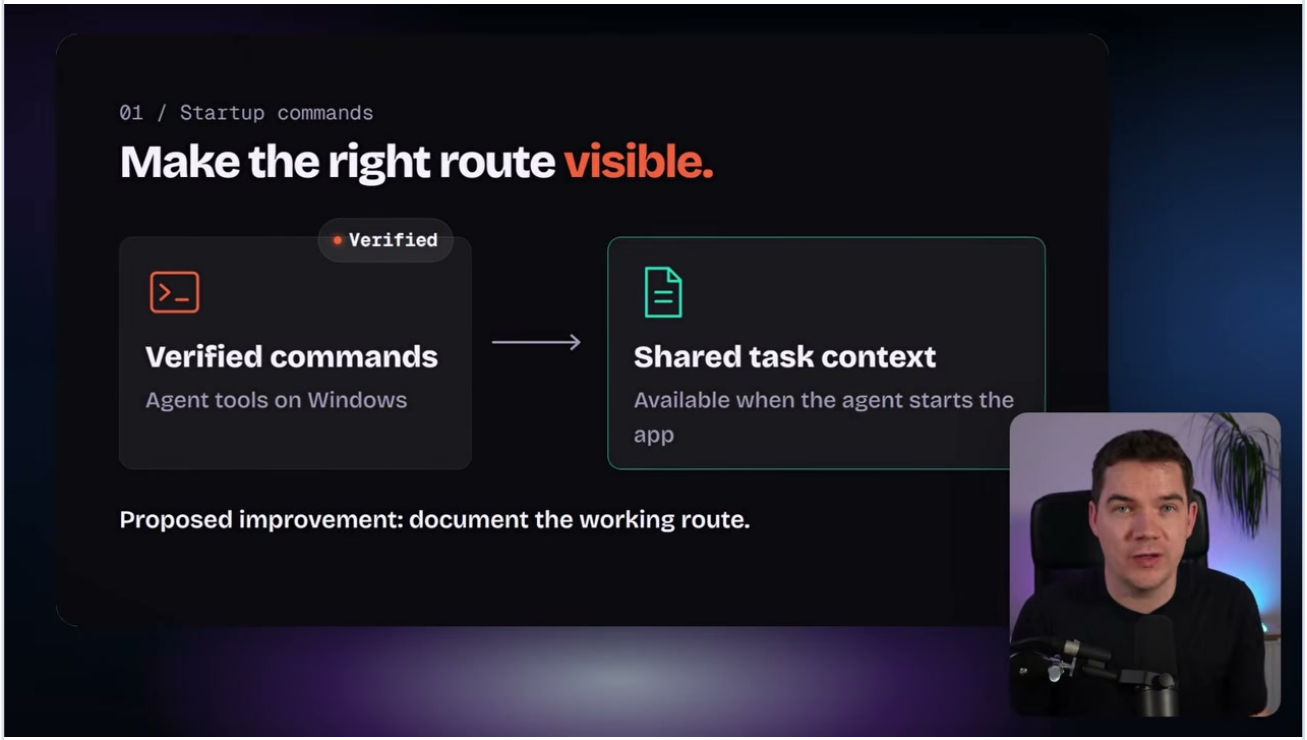
- Calling the script directly was not recognized, while another route hit a script-execution restriction.
- The illustrated working route uses PowerShell's -File option and reaches an app-started state.
- The issue appeared often enough in the sampled sessions to be flagged, though not in every run.

WHY IT MATTERS

Ambiguous bootstrap instructions spend session time and tokens before feature work even begins.

Proposed Change — Make the Route Visible

Move verified commands into shared task context



SPEAKER NOTES

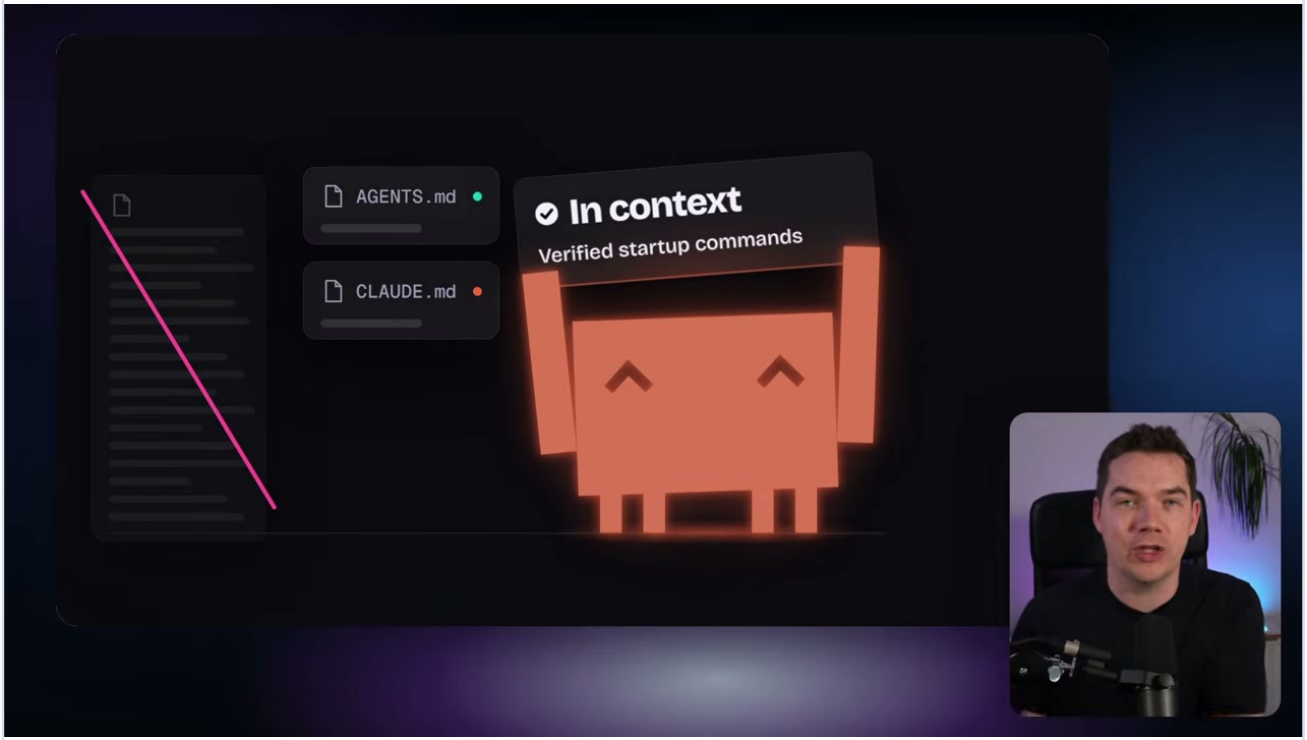
- The proposed improvement is to document the verified startup route where the agent can retrieve it when needed.
- The goal is shared task context, not a large instruction file loaded into every interaction.
- The user should review the command and context placement before changing project instructions.

WHY IT MATTERS

A known-good entry path removes repeated discovery work while keeping instructions intentional.

Keep Startup Guidance in Context

AGENTS.md or CLAUDE.md are options, but keep them lean



SPEAKER NOTES

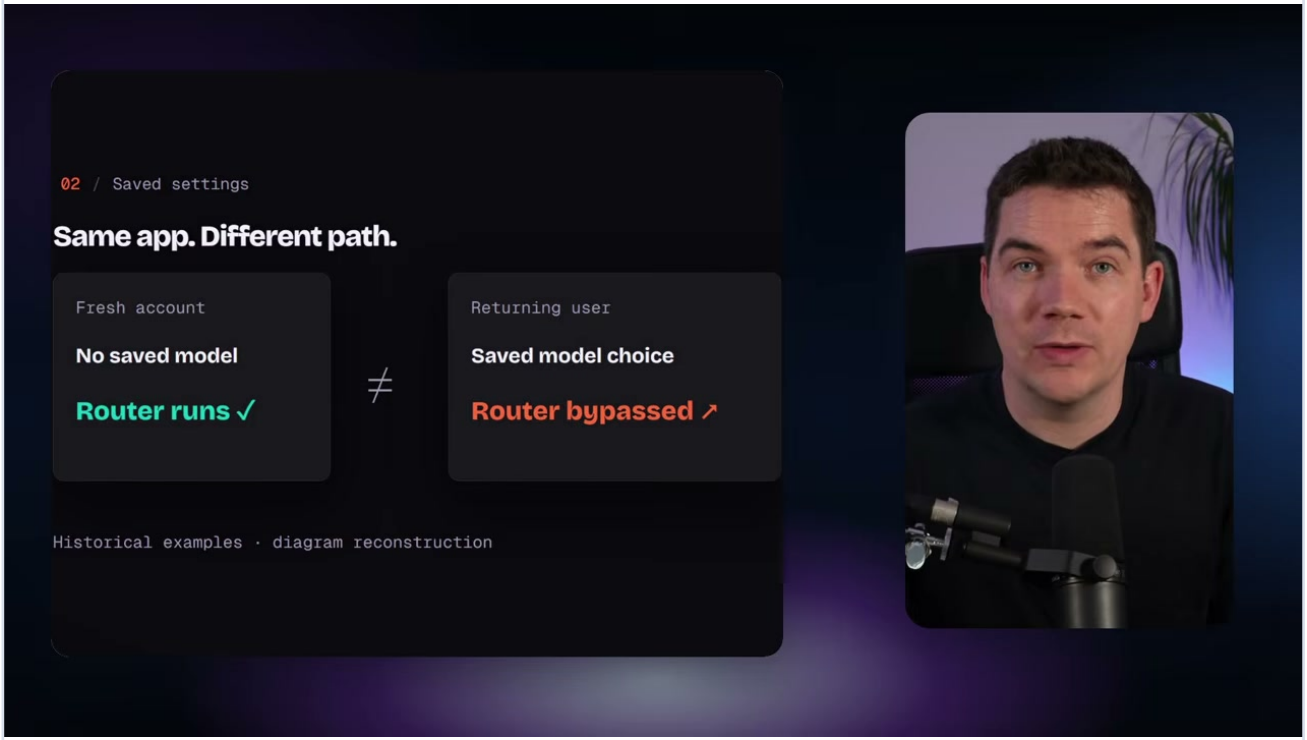
- The visual places verified startup commands beside AGENTS.md and CLAUDE.md as possible context sources.
- The transcript cautions that these shared instruction files should stay lean.
- A user should choose the smallest appropriate file and review the wording before adoption.

WHY IT MATTERS

Instructions help only when they are available at the moment of action without bloating every prompt.

Finding 2 — Same App, Different State

A saved model choice bypassed the new router



SPEAKER NOTES

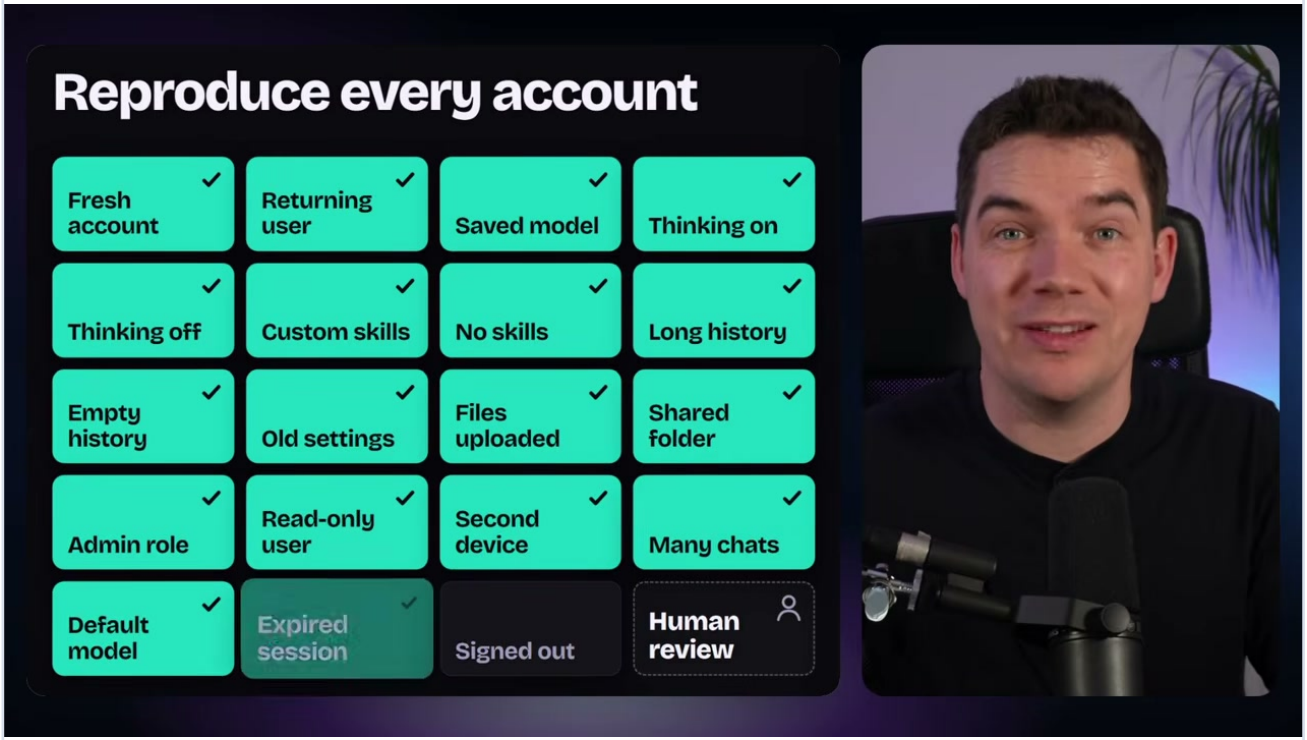
- A fresh test account had no saved model, so the new routing path ran successfully.
- The presenter's returning account carried a saved model choice and bypassed that path.
- This causal reconstruction comes from the project's observed account states, not a general routing benchmark.

WHY IT MATTERS

A feature can pass on a clean account yet fail for real users whose persistent settings select another path.

Proposed Change — Reproduce Real Account States

Expand coverage beyond the clean test account



SPEAKER NOTES

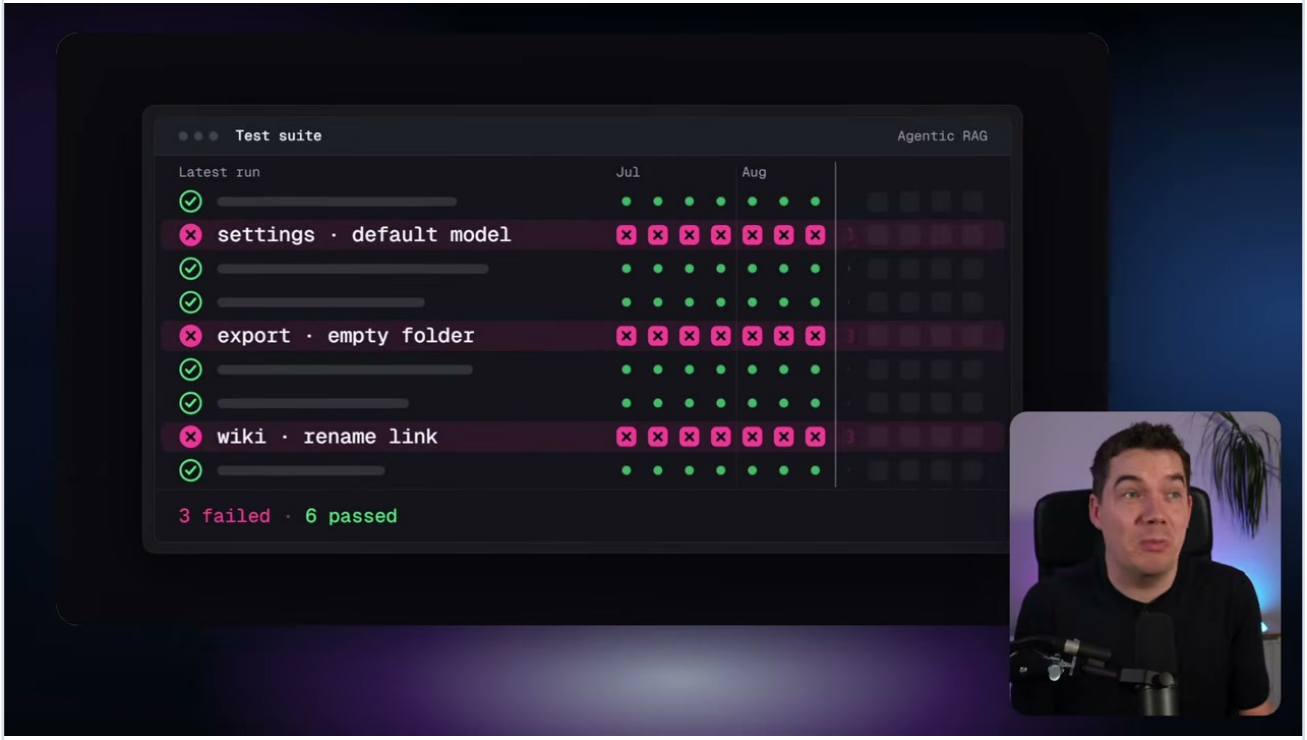
- The matrix spans fresh and returning accounts, saved settings, thinking modes, skills, history, roles, devices, files, sessions, and model choices.
- The recommendation is to validate relevant configurations before handing the feature to a human reviewer.
- Teams should review and prioritize the matrix for their own risk profile before adding tests or environment variants.

WHY IT MATTERS

State-aware testing catches paths that idealized test fixtures never exercise.

Finding 3 — Old Tests Already Failed

Three persistent failures weakened the test signal



SPEAKER NOTES

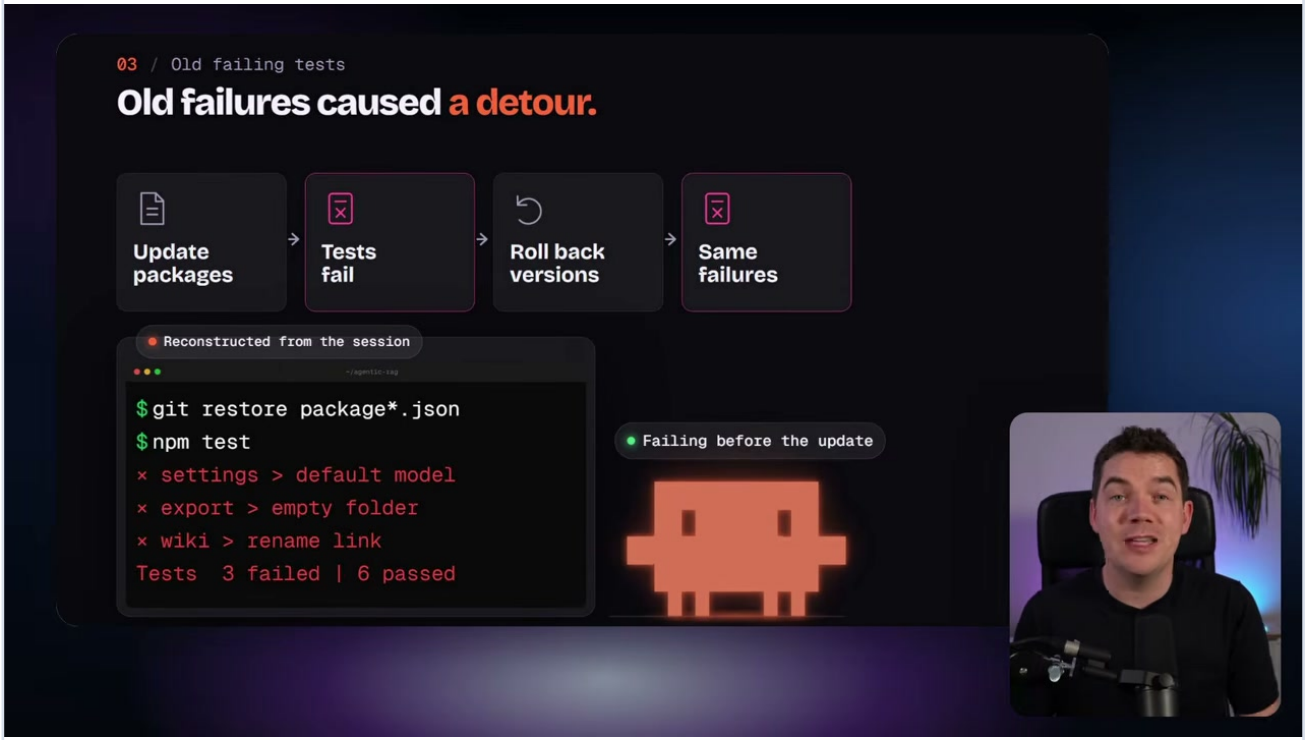
- The illustrated suite shows three failures and six passes, with failures recurring in historical runs.
- The presenter says the minor failures came from test definitions rather than current feature behavior.
- This was a lingering condition in this codebase, not evidence that old tests generally fail.

WHY IT MATTERS

A permanently noisy suite forces agents to investigate whether every known failure is a new regression.

The False Regression Detour

Update, fail, roll back, and see the same failures



SPEAKER NOTES

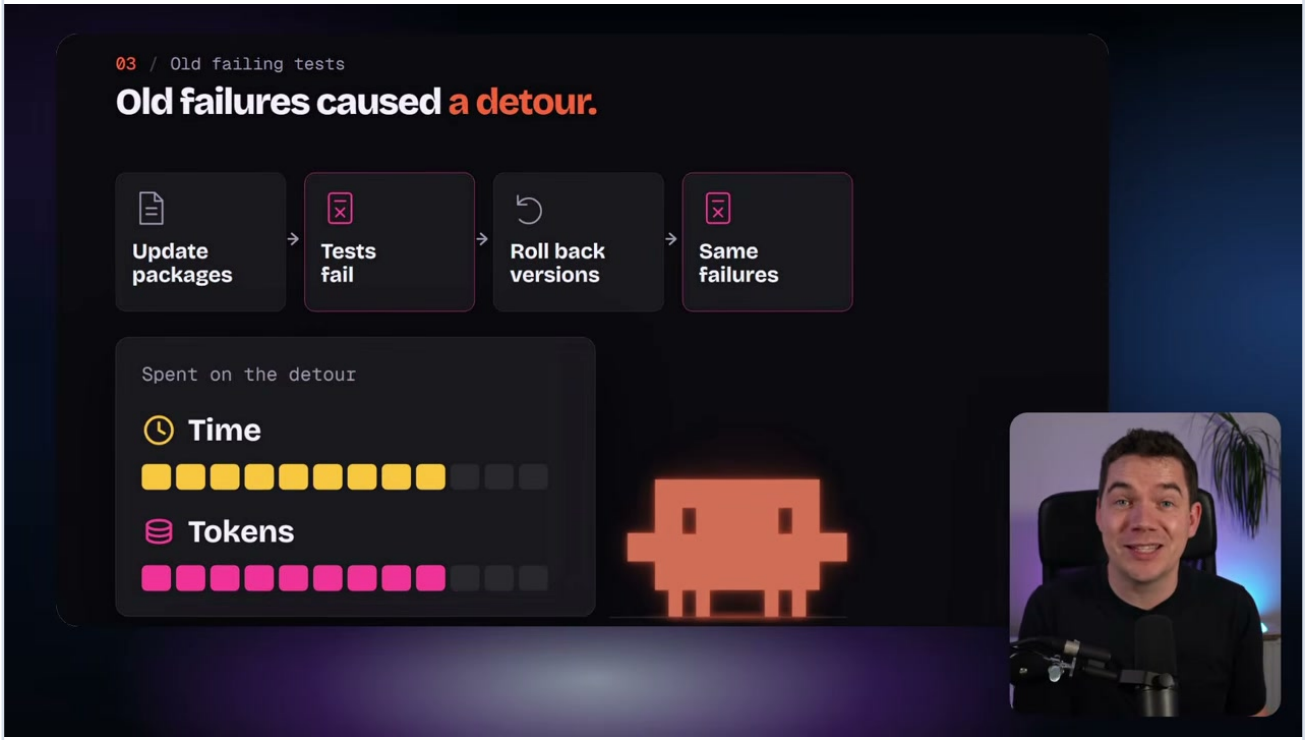
- After updating packages, the agent ran tests and encountered the three known failures.
- It restored the earlier package versions and reran the suite, but the same failures remained.
- The reconstruction shows that the failures predated the update, so the rollback was a false lead.

WHY IT MATTERS

Historical test context would have prevented a needless diagnostic loop.

The Detour Spent Time and Tokens

Noisy validation has an operational cost



SPEAKER NOTES

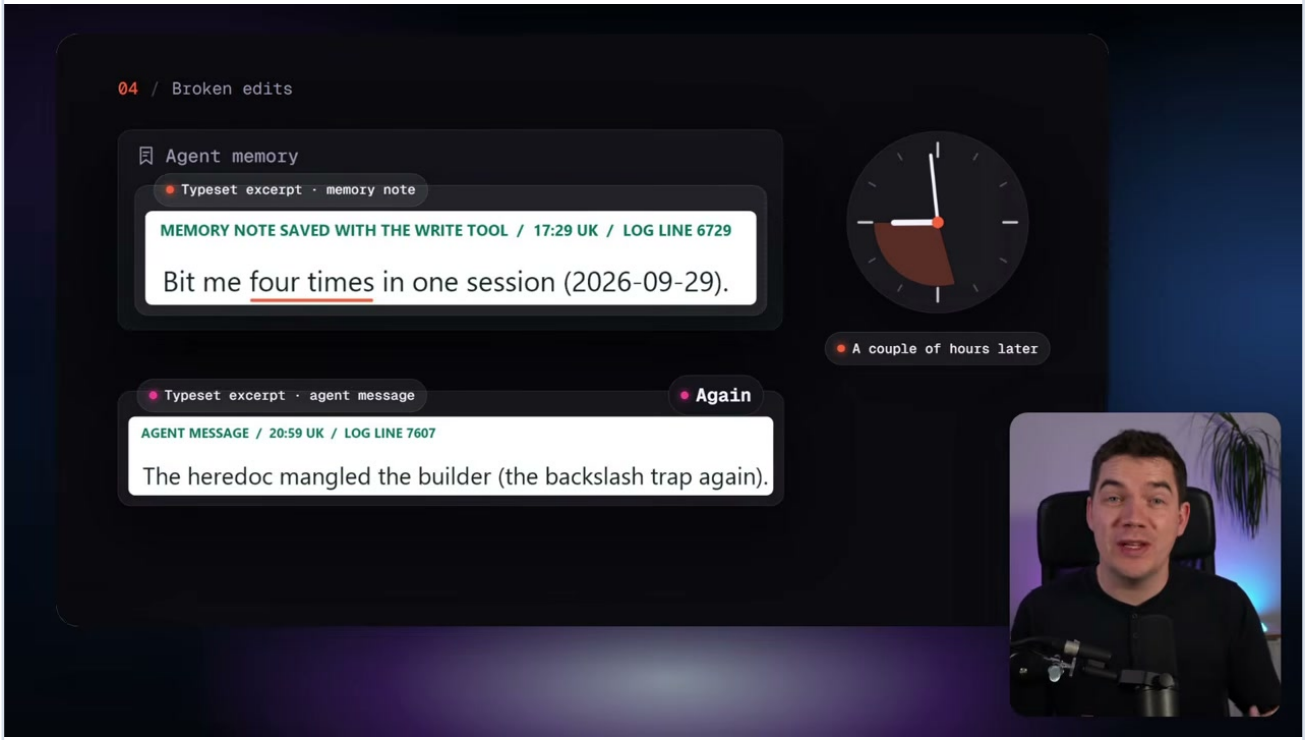
- The visual explicitly attributes both elapsed time and token usage to the detour.
- No exact quantities are reported, so the cost is directional rather than measured here.
- The proposed response is to repair the broken tests, subject to normal user review of code changes.

WHY IT MATTERS

Reliable tests let failures function as warnings instead of mysteries that consume an agent's budget.

Finding 4 — Memory Did Not Stop the Repeat

A destructive editing pattern returned after being recorded



SPEAKER NOTES

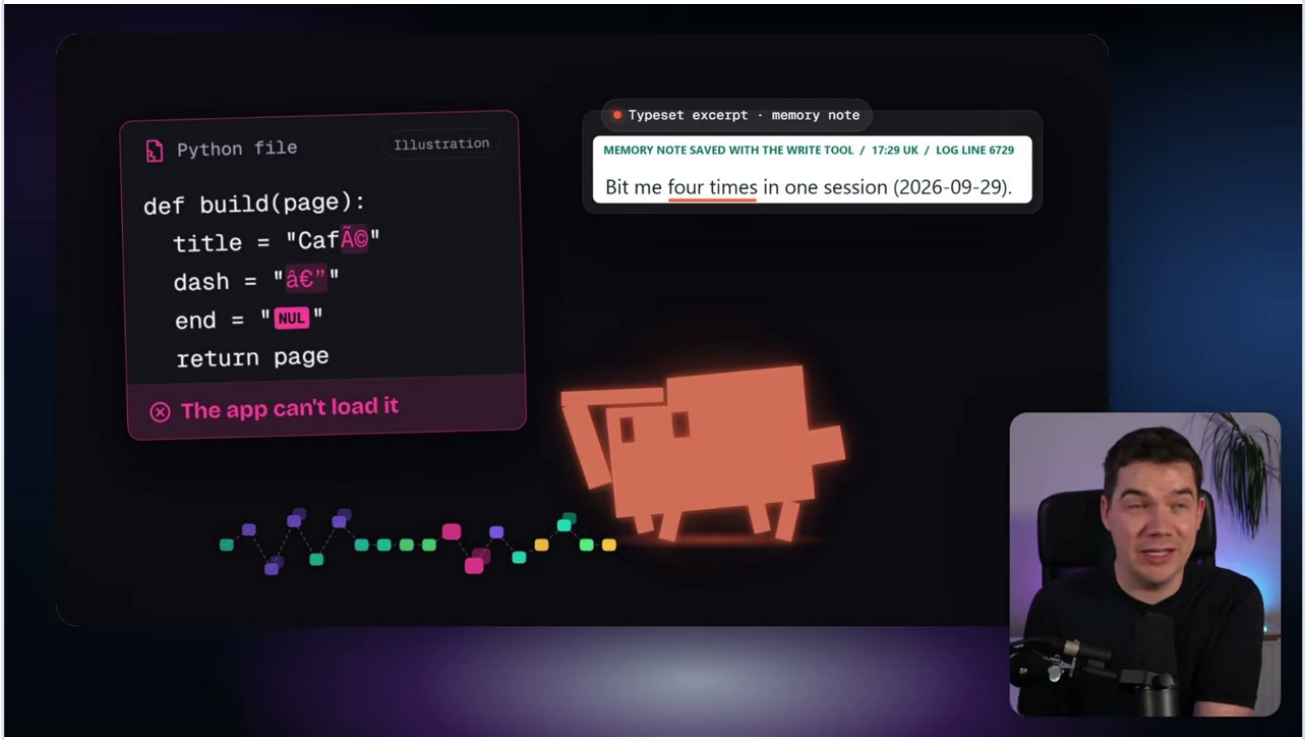
- The agent saved a warning after the problem occurred four times in one session.
- A later note says a heredoc mangled the builder through the backslash trap again.
- The example is a failure from this session history, not proof that agent memory never works.

WHY IT MATTERS

A reminder is weaker than an enforceable guard when the same edit can corrupt files.

The Edit Corrupted the File

Special characters were converted and the app could not load



SPEAKER NOTES

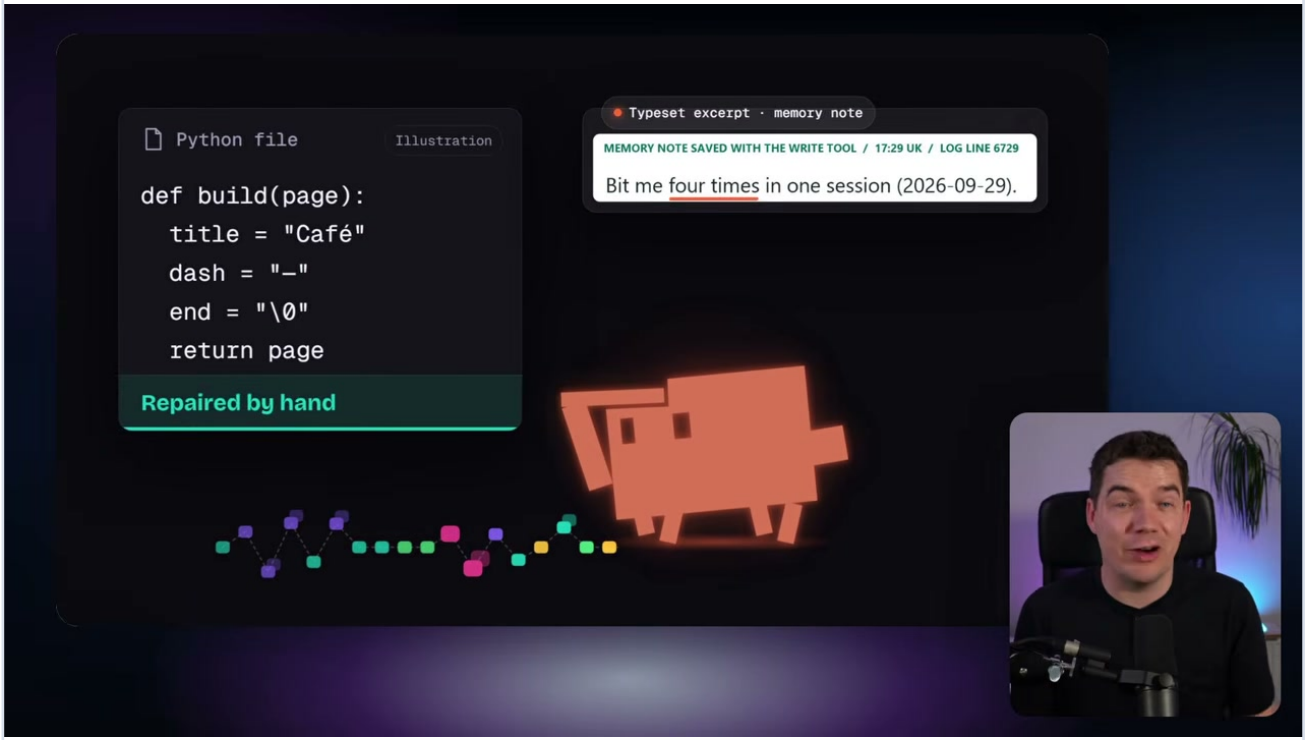
- The reconstruction shows a script-based edit turning special characters into invalid or mangled text.
- The resulting Python file is marked with the outcome: the app cannot load it.
- The feature task had to pause while the agent repaired its own changes.

WHY IT MATTERS

A syntactically or encoding-damaged edit can turn local automation into codebase damage.

Recovery Required Manual Repair

The file was restored after the failed edit



SPEAKER NOTES

- The visual ends the incident with the file marked repaired by hand.
- Manual recovery adds work that is absent from the final feature summary.
- The need for repair was observed in this project session and should not be generalized without more evidence.

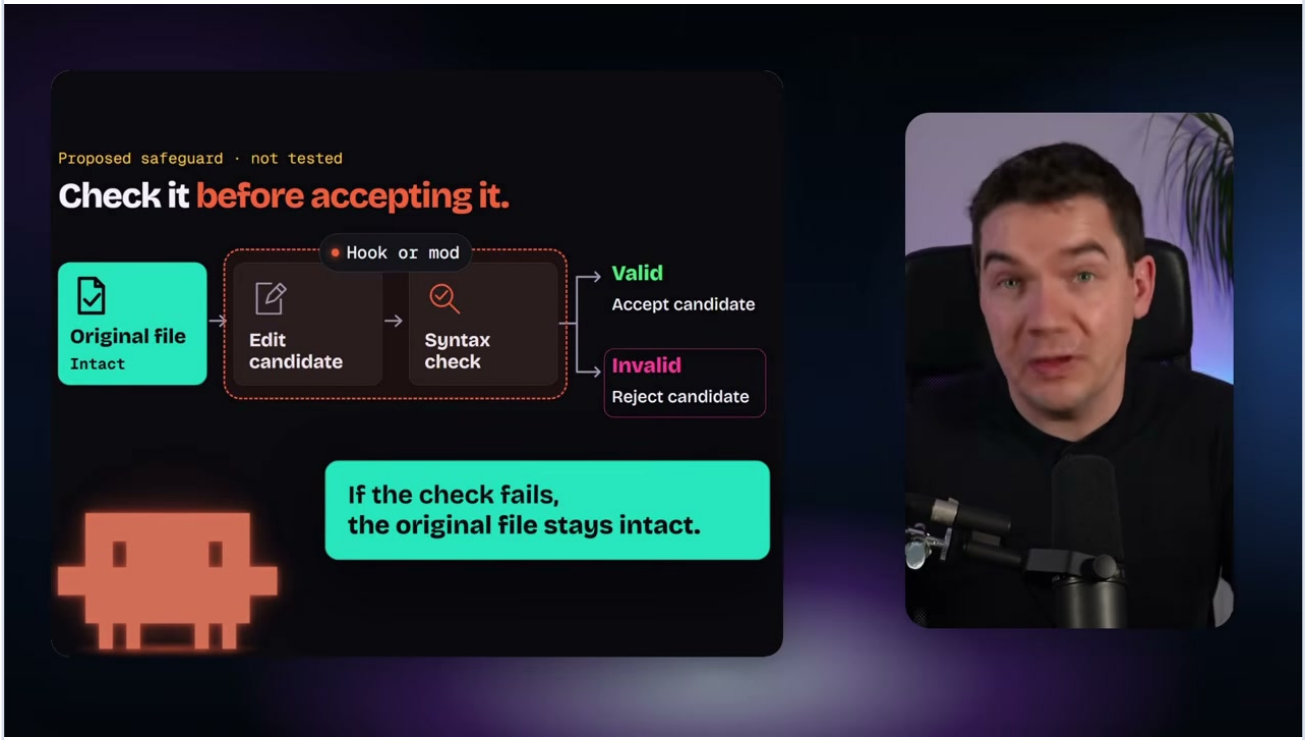
WHY IT MATTERS

Retrospectives reveal repair work that completion metrics can hide.

Source: https://www.youtube.com/watch?v=7nNJZmyBlm0

Proposed Safeguard — Validate Before Accepting

Reject a bad candidate while preserving the original file



SPEAKER NOTES

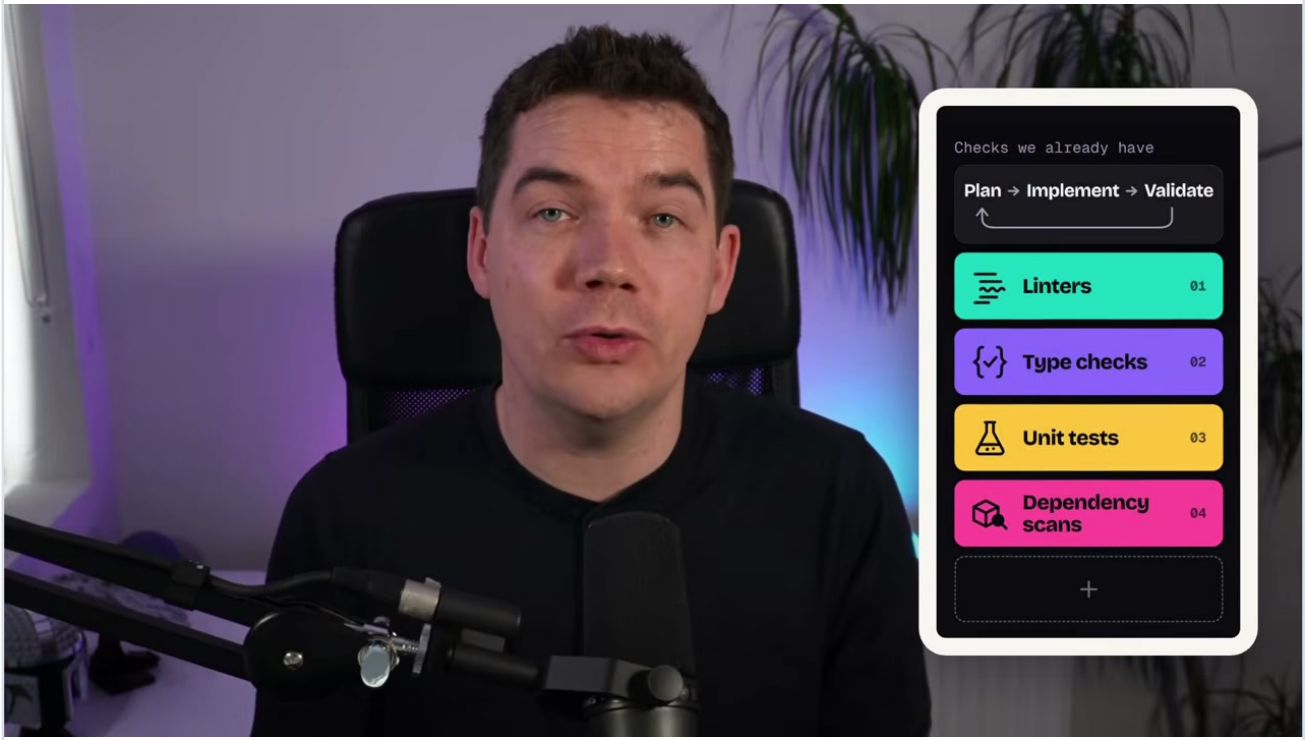
- The proposed flow snapshots the original, applies an edit candidate, and runs a syntax check.
- Valid candidates are accepted; invalid candidates are rejected so the original remains intact.
- The slide explicitly labels this safeguard not tested, and the user must review it before changing hooks, mods, or code.

WHY IT MATTERS

An automated acceptance gate can turn a remembered warning into a mechanical safety property.

Retrospectives Add Another Health Check

They complement linters, types, tests, and dependency scans



SPEAKER NOTES

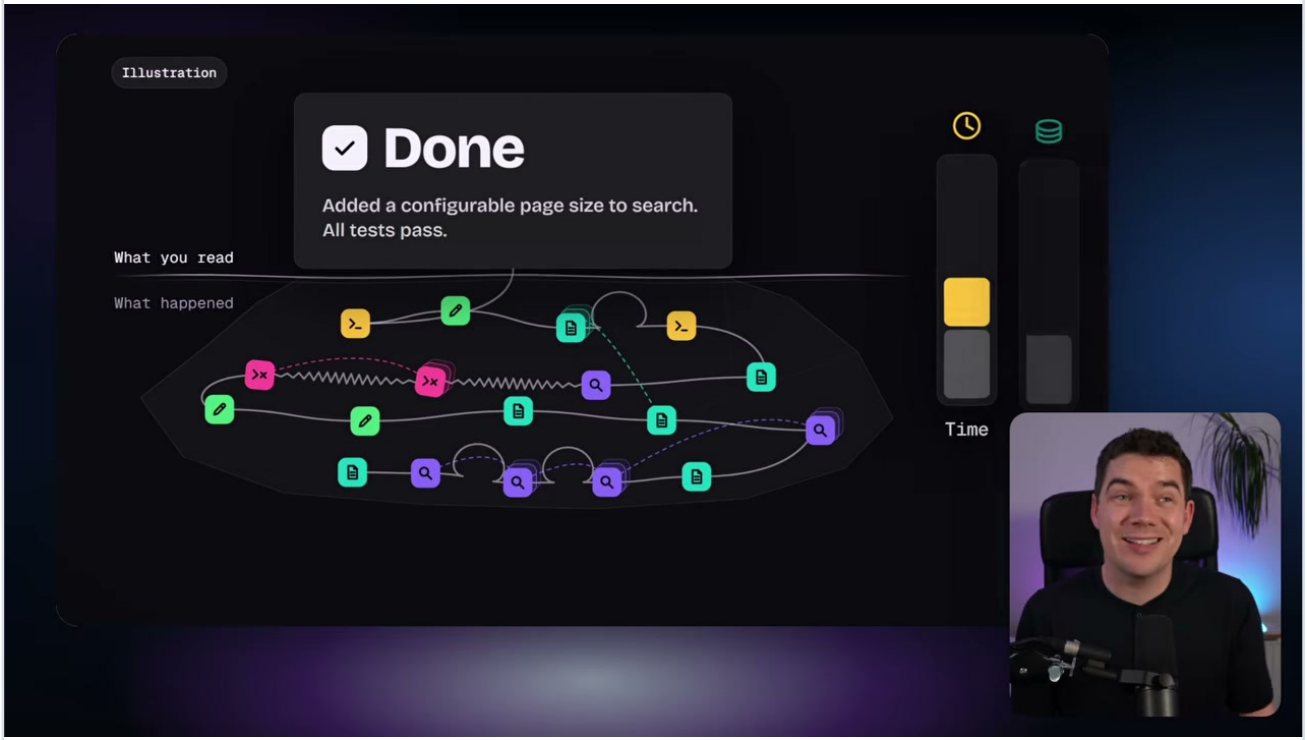
- Existing checks validate properties of the code and dependencies after implementation.
- Session analysis asks a different question: how did the agent reach the result?
- The layers are complementary; /retro is not presented as a replacement for conventional validation.

WHY IT MATTERS

Process evidence closes a blind spot left by checks that examine only the resulting artifact.

Done and Passing Is Not the Whole Story

The result card sits above the full execution trace



SPEAKER NOTES

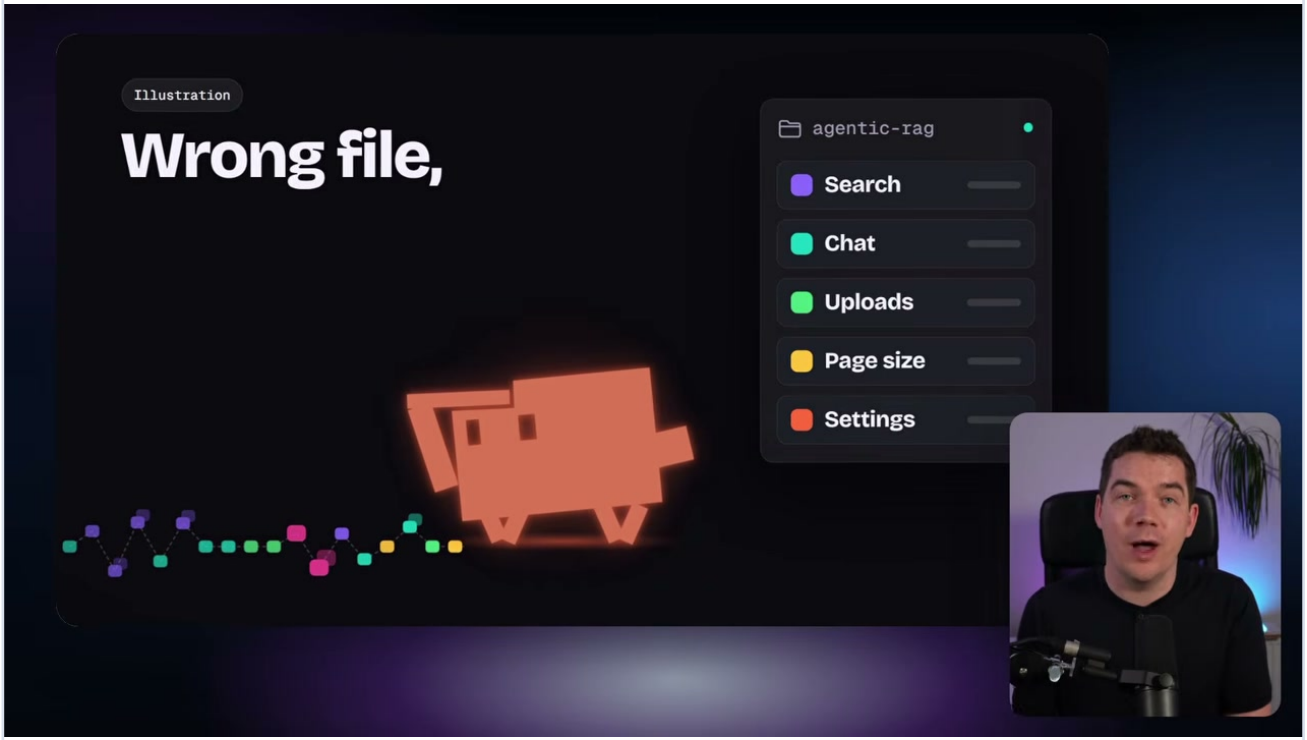
- The illustrated task reports a configurable page size and all tests passing.
- The trace beneath it distinguishes the summary the user reads from what happened during execution.
- A successful endpoint can coexist with avoidable file searches, retries, or workarounds.

WHY IT MATTERS

Reviewing execution evidence prevents a clean final message from hiding inefficient or risky behavior.

A Wrong File Can Still Produce Activity

Misguided context can degrade the codebase despite apparent progress



SPEAKER NOTES

- The warning illustrates an agent working in the wrong file or project area.
- The transcript notes that missing the right document or file can lead to duplication or lower-quality changes.
- This is a general risk illustrated by the talk, not a quantified failure rate.

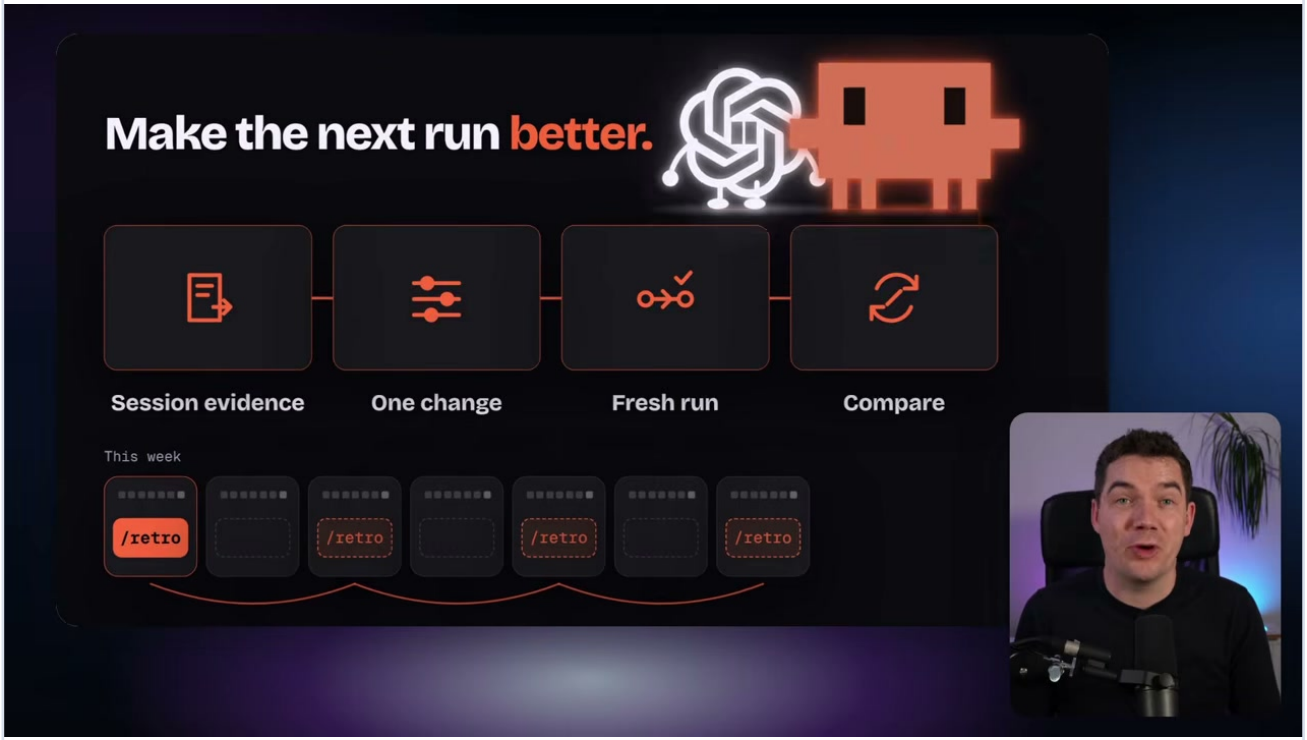
WHY IT MATTERS

Outcome checks may pass while poor context selection creates maintenance debt elsewhere.

Source: https://www.youtube.com/watch?v=7nNJZmyBlm0

Make the Next Run Better

Use evidence, change one thing, rerun fresh, and compare



SPEAKER NOTES

- The closing loop begins with session evidence and proposes one targeted change.
- A fresh run reduces contamination from the previous state, then comparison tests whether the change helped.
- Any instruction, environment, or codebase change still requires user review before the experiment.

WHY IT MATTERS

Retrospectives become useful when findings feed a controlled improvement loop rather than an unchecked rewrite.

Source: https://www.youtube.com/watch?v=7nNJZmyBlm0